Game Optimization

Alpha–Beta Pruning · Evaluation Functions · Problems in Game Playing

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

Today’s Content

- **Recap: Minimax and its cost**
- **Alpha–Beta pruning: the idea**
- **Alpha–Beta — step by step**
- **Algorithm and move ordering**
- **Evaluation functions and cutting off search**
- **Problems in game playing (and their fixes)**
- **Alpha–Beta for Tic-Tac-Toe in Python**

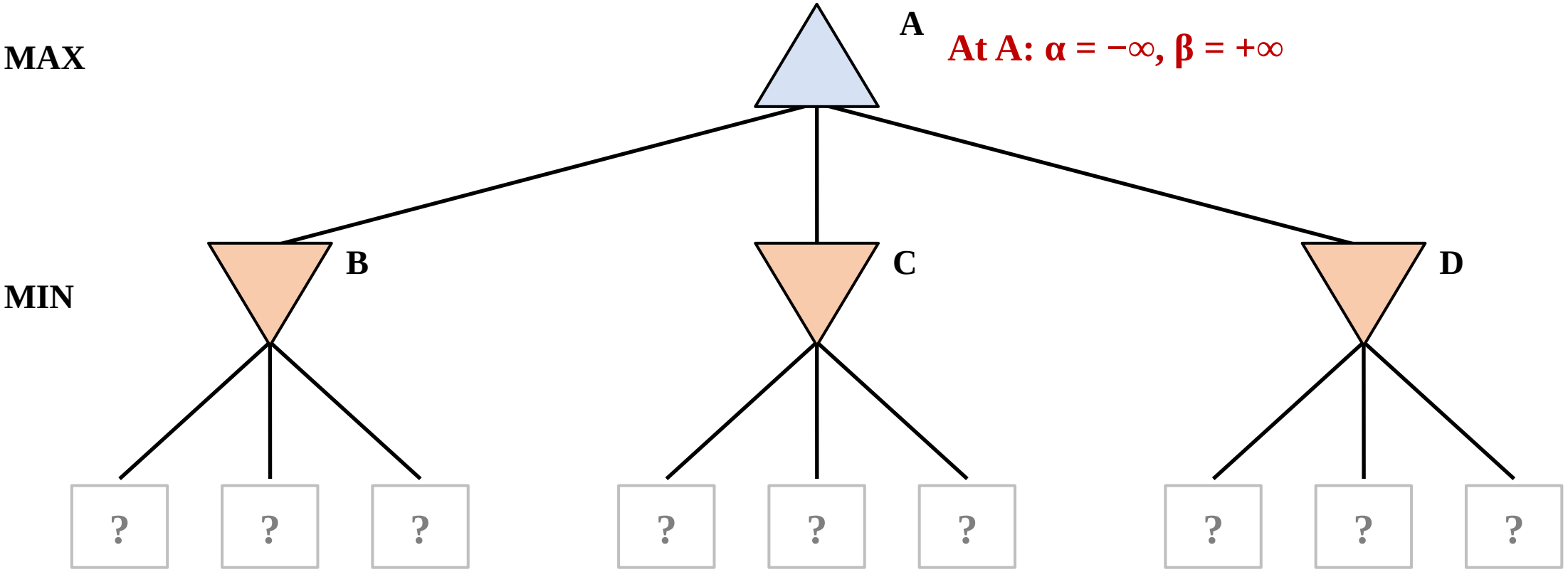
Recap: Minimax Is Too Slow

- **Minimax: MAX takes the highest value, MIN the lowest; values are computed bottom-up.**
- **It visits every node of the game tree: time $O(b^m)$.**
- **Tic-Tac-Toe: 549,946 nodes from the empty board (counted by our Python program).**
- **Chess: about 35^{100} nodes — impossible.**
- **Two ideas today:**
  - **Alpha–Beta pruning: skip branches that cannot change the decision**
  - **Evaluation functions: stop early and estimate the value**

Alpha–Beta Pruning: The Idea

- **Same answer as minimax, but it does not look at branches that cannot affect the final decision.**
- **It keeps two numbers while searching:**
  - **α (alpha) = the best value MAX can already guarantee on this path (starts at −∞)**
  - **β (beta) = the best value MIN can already guarantee on this path (starts at +∞)**
- **Prune when α ≥ β: the opponent would never let the game reach this branch.**
- **Pruning never changes the result — it only saves work.**

Alpha–Beta: Step by Step (0/5)

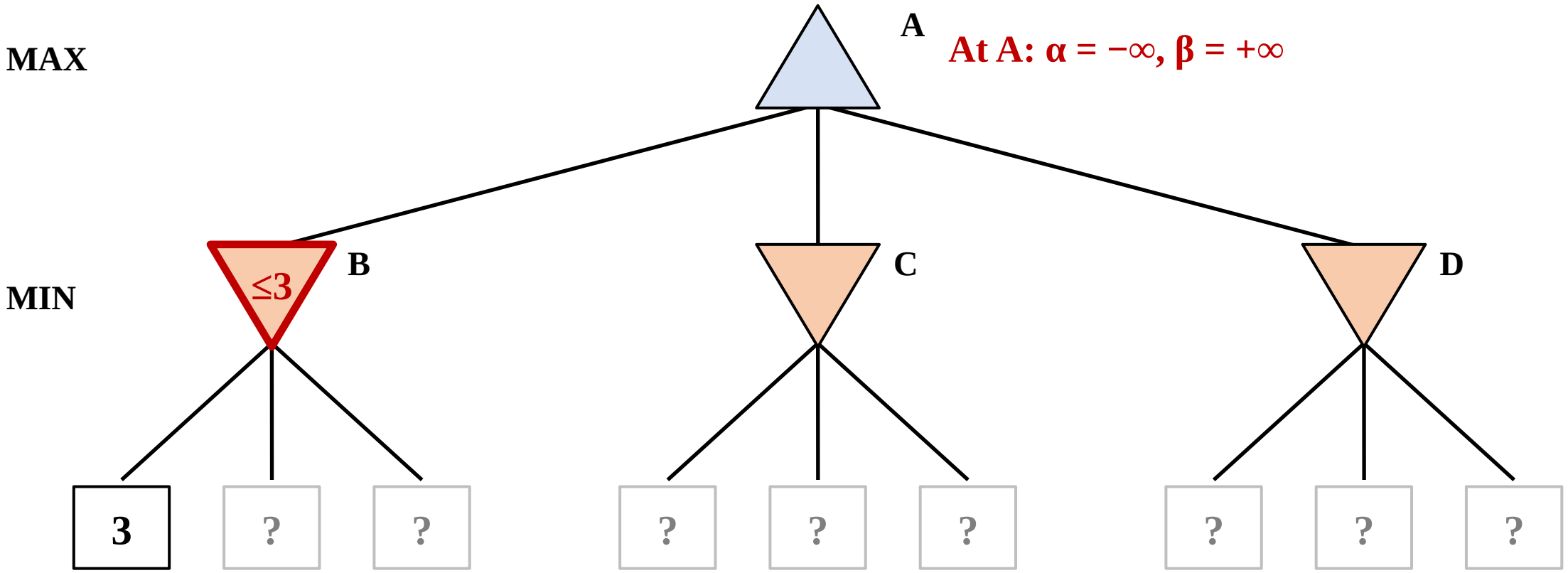


Start

Same tree as last class. A is MAX; B, C, D are MIN. Leaves are hidden until we look at them.

MAX (picks largest) **MIN (picks smallest)** **chosen move** ✂ **= pruned (never looked at)**

Alpha–Beta: Step by Step (1/5)

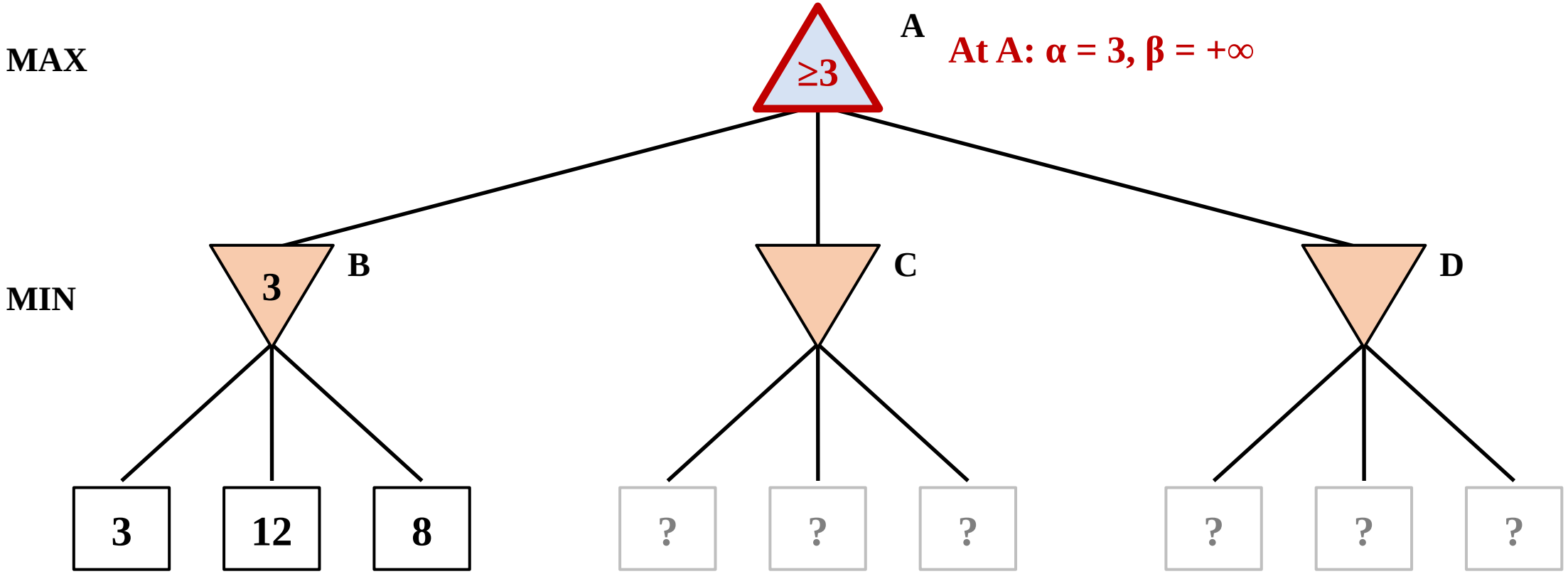


Step 1: look at B's first leaf: 3

B is MIN, so B will be at most 3 (B ≤ 3).

MAX (picks largest) MIN (picks smallest) chosen move ✂ = pruned (never looked at)

Alpha–Beta: Step by Step (2/5)

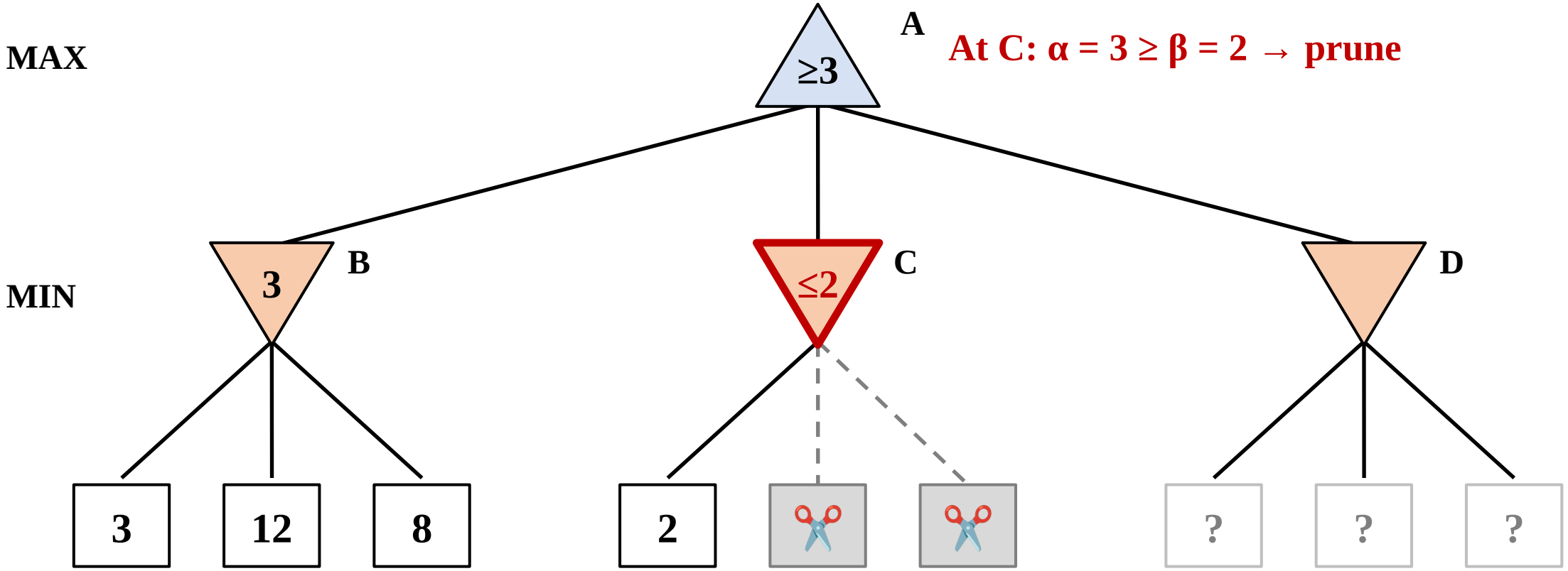


Step 2: B's other leaves 12, 8 → B = 3

MIN picks 3. Back at A: MAX can already guarantee 3, so α = 3 (A ≥ 3).

MAX (picks largest) **MIN (picks smallest)** **chosen move** ✂ **= pruned (never looked at)**

Alpha–Beta: Step by Step (3/5)

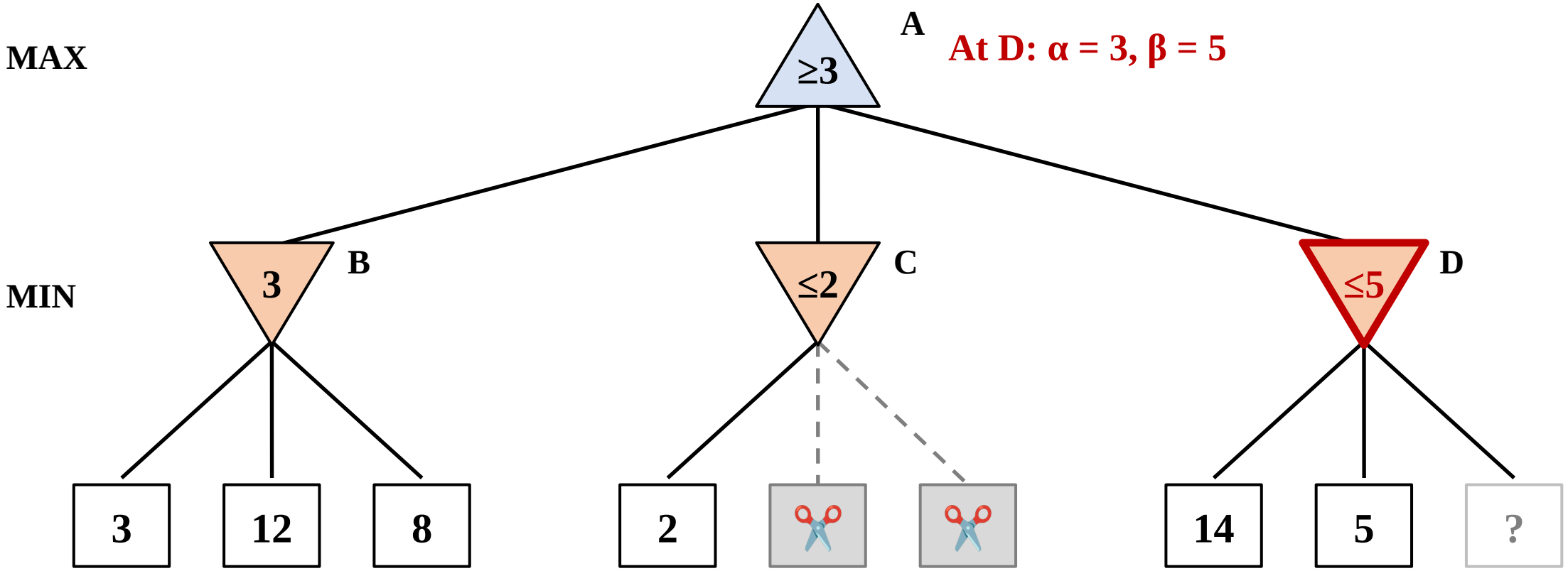


Step 3: C's first leaf is 2 → C ≤ 2 → PRUNE ✂

C can be at most 2, but MAX already has 3 from B. MAX will never choose C, so 4 and 6 are never examined.

MAX (picks largest) **MIN (picks smallest)** **chosen move** **✂ = pruned (never looked at)**

Alpha–Beta: Step by Step (4/5)

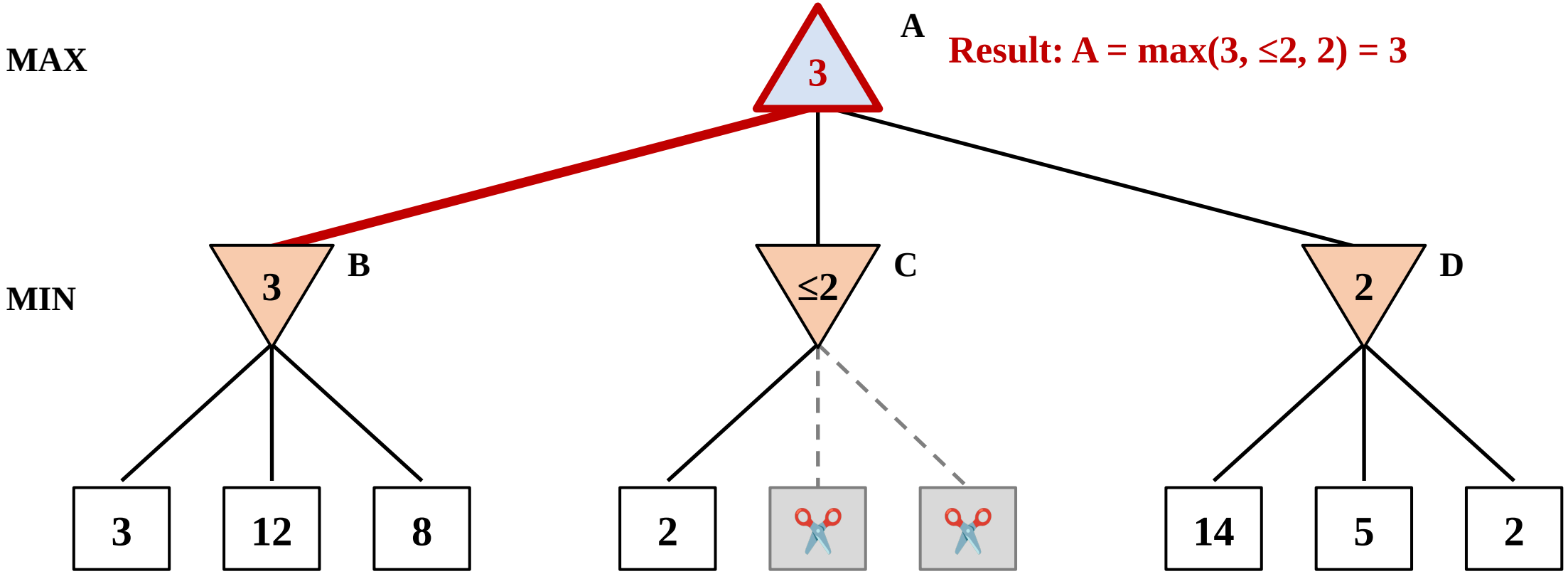


Step 4: D's leaves 14, 5 → D ≤ 5

Still bigger than α = 3, so D might be better than B — keep looking.

MAX (picks largest) MIN (picks smallest) chosen move ✂ = pruned (never looked at)

Alpha–Beta: Step by Step (5/5)



Step 5: D's last leaf 2 → D = 2 → A = 3, play a1 ✓

Same answer as minimax (3, move a1), but 2 of the 9 leaves were never examined.

MAX (picks largest) MIN (picks smallest) chosen move ✂ = pruned (never looked at)

The Alpha–Beta Algorithm

```
function MAX-VALUE(state, α, β):
   if TERMINAL-TEST(state): return UTILITY(state)
   v ← −∞
   for each action a:
      v ← max(v, MIN-VALUE(RESULT(state, a), α, β))
      if v ≥ β: return v          // prune: MIN will avoid this
      α ← max(α, v)
   return v

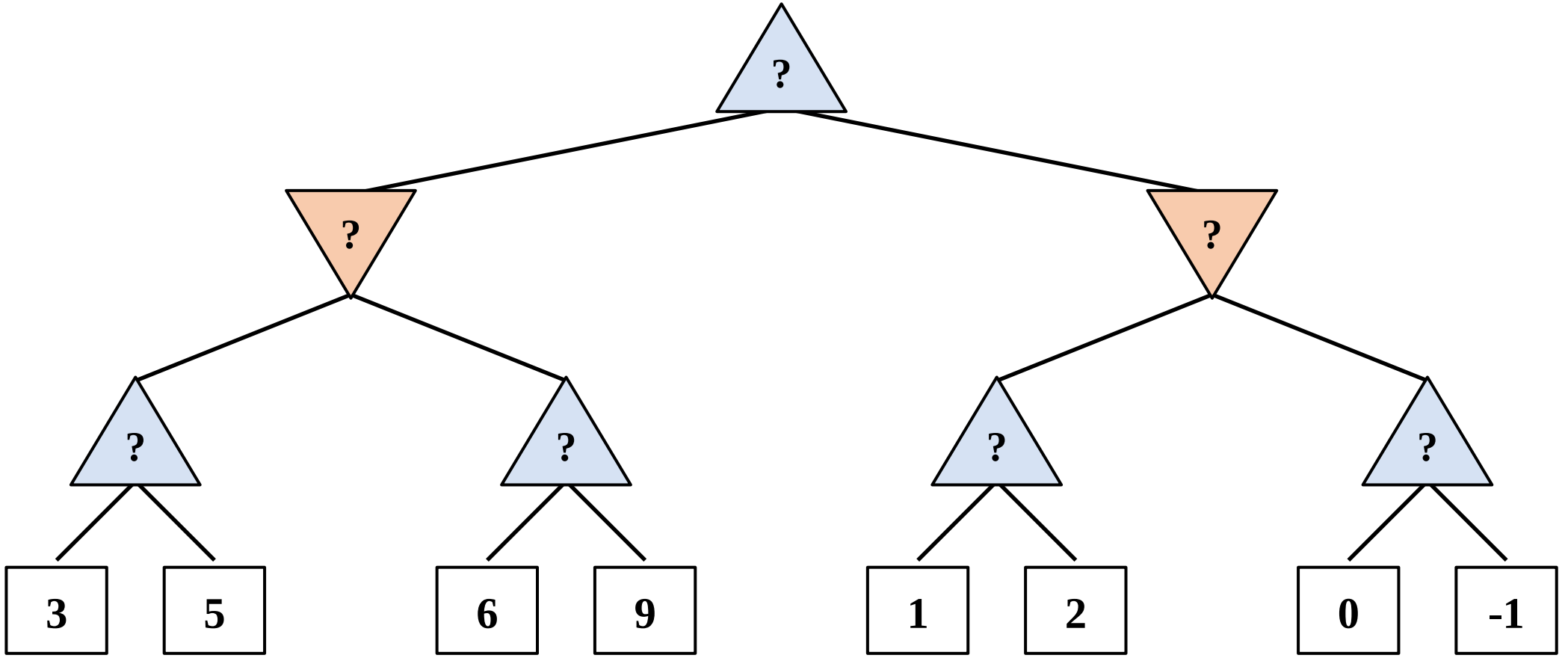
function MIN-VALUE(state, α, β):
   if TERMINAL-TEST(state): return UTILITY(state)
   v ← +∞
   for each action a:
      v ← min(v, MAX-VALUE(RESULT(state, a), α, β))
      if v ≤ α: return v          // prune: MAX will avoid this
      β ← min(β, v)
   return v
```

Move Ordering Matters

Order of moves	Nodes examined	Effect
Best moves first (perfect ordering)	$O(b^{m/2})$	Can search about TWICE as deep in the same time
Random order	about $O(b^{3m/4})$	Still a big saving
Worst order	$O(b^{m})$	No pruning at all — same as minimax

- In our tree, if D's leaves came as 2, 5, 14: the first leaf gives $D \leq 2 < \alpha = 3 \rightarrow$ 5 and 14 are pruned too (4 of 9 leaves skipped).
- Good programs try promising moves first (captures, threats, the best move from a previous search).

Class Exercise: Which Branches Are Pruned?



Run alpha–beta from left to right. Which leaves or subtrees are never examined? Is the root value the same as minimax (5)?

MAX (picks largest) MIN (picks smallest) chosen move

Answer: Pruning in the Exercise Tree

Step	What we learn	Pruned?
Left: C = max(3, 5) = 5	Left MIN ≤ 5 (β = 5)	—
D: first leaf 6	D ≥ 6 ≥ β = 5 → MIN will never choose D	Leaf 9 ✂️
Root after left side	α = 5	—
Right: E = max(1, 2) = 2	Right MIN ≤ 2 ≤ α = 5 → MAX will never go right	Whole F subtree (0, −1) ✂️
Root	max(5, ≤ 2) = 5 → go LEFT	Same as minimax ✓

Part 2

Evaluation Functions

Stopping early and estimating

Cutting Off the Search

- **Even with alpha–beta, we cannot search to the end of a chess game.**
- **Instead: search to a depth limit d, then estimate how good the position is.**
- **Replace TERMINAL-TEST with CUTOFF-TEST (depth limit reached) and UTILITY with EVAL (an estimate).**

Situation	H-MINIMAX(s, d)
CUTOFF-TEST(s, d) is true	EVAL(s)
MAX's turn	max over moves of H-MINIMAX(RESULT(s, a), d + 1)
MIN's turn	min over moves of H-MINIMAX(RESULT(s, a), d + 1)

Evaluation Functions

- **EVAL(s) = estimate of the expected utility from state s.**
- **Usually a weighted sum of features:**

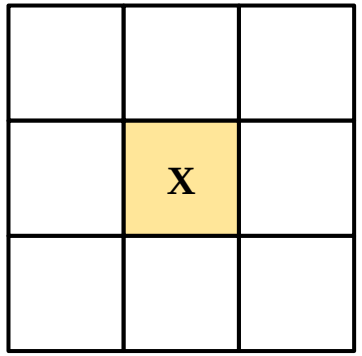
$$\mathbf{EVAL(s) = w1 \cdot f1(s) + w2 \cdot f2(s) + \ldots + wn \cdot fn(s)}$$

Chess piece	Pawn	Knight	Bishop	Rook	Queen
Material value	1	3	3	5	9

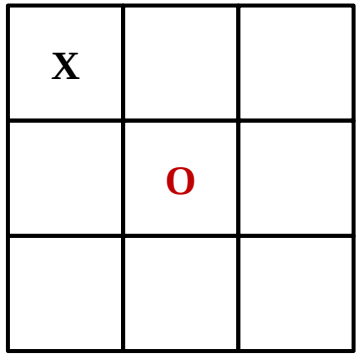
- **Example: White = Queen + 2 Pawns = 11; Black = Rook + Bishop + Pawn = 9 → EVAL = 11 − 9 = +2 (good for White).**
- **A good EVAL: correct at terminal states, fast to compute, and strongly linked to the real chance of winning.**

Example: An Evaluation Function for Tic-Tac-Toe

EVAL = (lines still open for X) − (lines still open for O)



Open for X: 8
Open for O: 4
EVAL = 8 − 4 = +4



Open for X: 4
Open for O: 5
EVAL = 4 − 5 = −1

A line is “open” for X if it has no O in it (and vice versa).

Part 3

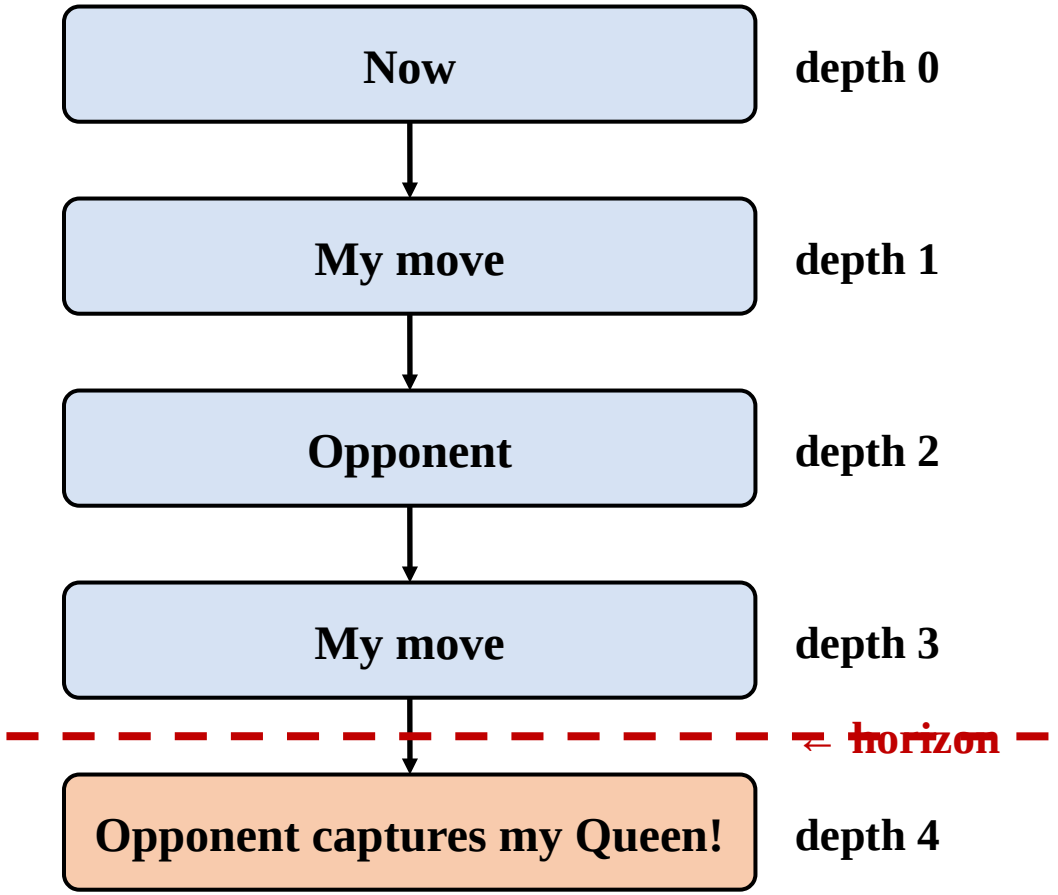
Problems in Game Playing

and how game programs deal with them

Problems in Game Playing

Problem	What goes wrong	Fix
Huge search space	Chess: about 35^{100} positions	Alpha–beta + depth limit + EVAL
Time limit per move	We must answer in, e.g., 30 seconds	Iterative deepening: depth 1, 2, 3 … until time runs out
Horizon effect	A disaster just beyond the depth limit is not seen	Search deeper on forced moves (singular extensions)
Non-quiet positions	EVAL is wrong in the middle of a capture exchange	Quiescence search: keep searching until the position is quiet
Repeated positions	Same position reached by different move orders	Transposition table (remember evaluated positions)
Opening and endgame	Searching is wasteful or too hard	Opening books and endgame databases

The Horizon Effect



- The search stops at depth 3: EVAL says “all fine”.
- The queen is lost at depth 4 — beyond the horizon.
- The program may even waste pawns to delay the loss past the horizon.
- Fixes: search deeper on forcing moves; quiescence search (do not stop in the middle of captures).

Alpha–Beta for Tic-Tac-Toe in Python

```
def alphabeta(b, player, alpha=-2, beta=2):
    w = winner(b)                  # winner() as in last class
    if w == 'X': return 1
    if w == 'O': return -1
    if ' ' not in b: return 0
    other = 'O' if player == 'X' else 'X'
    for i in range(9):
        if b[i] != ' ': continue
        v = alphabeta(b[:i] + player + b[i+1:], other, alpha, beta)
        if player == 'X': alpha = max(alpha, v)
        else:             beta = min(beta, v)
        if alpha >= beta: break      # prune the remaining moves
    return alpha if player == 'X' else beta

print(alphabeta(' ' * 9, 'X'))   # 0 (draw)
```

From the empty board	Positions visited	Value
Minimax	549,946	0
Alpha–Beta	18,297 (≈ 30× fewer)	0

Quick Check

- 1. What do α and β stand for?
- 2. When does alpha–beta prune a branch?
- 3. Can alpha–beta give a different answer from minimax?
- 4. Why does move ordering matter?
- 5. What is the horizon effect, and one way to reduce it?

Summary

- **Alpha–beta pruning gives the same result as minimax but skips branches that cannot matter ($\alpha \geq \beta$).**
- **With good move ordering it searches about twice as deep: $O(b^{m/2})$.**
- **Real games need a depth limit and an evaluation function (a weighted sum of features).**
- **Problems in game playing: huge search space, time limits, horizon effect, non-quiet positions — fixed with iterative deepening, quiescence search, transposition tables, opening books.**
- **Reading: Russell & Norvig, AIMA, 5.3–5.4.**
- **Next class: Propositional Logic & Knowledge Representation.**

Thank You

Any questions?